

Capítulo 17: Sistemas Distribuidos y Persistencia Reactiva

1. Explicación Teórica: Coherencia y Resiliencia a Gran Escala

El diseño de sistemas distribuidos y reactivos en Scala, típicamente utilizando Apache Pekko (el *fork* de código abierto del *toolkit* Akka), se centra en la capacidad de escalar la concurrencia a través de múltiples máquinas y garantizar que el **estado interno de las entidades de negocio no se pierda** ante fallos (persistencia reactiva).

A. Sistemas Distribuidos con Cluster Sharding

Cuando un sistema necesita gestionar miles o millones de entidades con estado (como sesiones de usuario o agregados de Domain-Driven Design), una sola máquina (nodo) es insuficiente. **Cluster Sharding** (Fragmentación de Clúster) es la solución de Pekko para este problema de escalamiento horizontal, permitiendo distribuir estos actores con estado (*entidades*) a través de múltiples nodos.

- **Abstracción de Localización:** El desarrollador envía un mensaje a una entidad utilizando su **identificador lógico** (ej., `userId` o `PersistenceId`), sin necesidad de conocer su ubicación física en el clúster.
- **Principio Single-Writer:** El *Cluster Sharding* es fundamental para la persistencia, ya que garantiza que solo **una instancia activa** del actor (entidad) para un `PersistenceId` dado exista en un momento dado en todo el clúster. Esto es vital para prevenir escrituras de eventos intercaladas que podrían corromper el estado.

B. Persistencia Reactiva con Event Sourcing

La **Persistencia Pekko** es el módulo que permite a los actores recuperar su estado tras un reinicio (debido a un fallo, una caída de nodo, o una supervisión que ordenó un `restart`). El patrón principal para lograr esto es **Event Sourcing** (Eventos Fuente).

- **Eventos Inmutables:** En lugar de guardar el estado actual (modelo CRUD), el actor persistente almacena una secuencia inmutable de **eventos** que representan todos los cambios de estado que han ocurrido. Los eventos se persisten añadiéndolos al almacenamiento (*journal*), y nunca se mutan.
- **Recuperación por Replay:** Si el actor se reinicia, su estado se **reconstruye reejecutando (replay) todos los eventos** almacenados en el *journal*, o desde el último *snapshot* (punto de control) si existe. Esto garantiza que, incluso después de un fallo, la nueva instancia del actor recupere su memoria con precisión forense.

2. Sintaxis: EventSourcedBehavior y Cluster Sharding

A. Sintaxis de EventSourcedBehavior (Persistencia de Eventos)

La lógica central para el estado persistente se define mediante el

`EventSourcedBehavior[Comando, Evento, Estado]`, el cual requiere cuatro componentes principales para su tipado seguro:

```
1  import org.apache.pekko.persistence.typed.scaladsl.{EventSourcedBehavior,
   Effect}
2  import org.apache.pekko.persistence.typed.PersistenceId
3
4  // 1. Comando, Evento, Estado (Modelos de Datos Inmutables/ADTs)
5  object EntidadCuenta {
6    // Estado Inmutable (lo que el actor recuerda)
7    final case class Estado(balance: BigDecimal)
8
9    // Comandos (lo que el actor recibe y valida)
10   sealed trait Comando
11   final case class Depositar(monto: BigDecimal, replyTo:
   ActorRef[Respuesta]) extends Comando
12
13   // Eventos (lo que se persiste)
14   sealed trait Evento
15   final case class Depositado(monto: BigDecimal) extends Evento
16
17   // ... Respuesta ...
18
19   // 2. Lógica para manejar Comandos -> Produce Efectos (Persistencia)
20   val commandHandler: (Estado, Comando) => Effect[Evento, Estado] =
21     (estado, comando) => comando match {
22       case Depositar(monto, replyTo) =>
23         // Persiste el evento antes de actualizar el estado o responder
24         Effect.persist(Depositado(monto))
25         // Efecto secundario: Responder solo después de la
   persistencia exitosa
26         .thenReply(replyTo)(_ => StatusReply.Success("Depósito
   exitoso"))
27     }
28
29   // 3. Lógica para aplicar Eventos -> Produce Nuevo Estado
30   val eventHandler: (Estado, Evento) => Estado =
31     (estado, evento) => evento match {
32       // SOLO actualización de estado, NUNCA efectos secundarios
33       case Depositado(monto) => estado.copy(balance = estado.balance +
   monto)
34     }
```

```

35
36 // 4. Comportamiento Raíz
37 def apply(id: String): Behavior[Comando] =
38   EventSourcedBehavior[Comando, Evento, Estado](
39     persistenceId = PersistenceId.ofUniqueId(id), // ID único
40     emptyState = Estado(BigDecimal(0)),          // Estado inicial
41     commandHandler = commandHandler,
42     eventHandler = eventHandler
43   )
44 }

```

B. Sintaxis de Cluster Sharding (Distribución)

El *Cluster Sharding* se inicializa registrando el `Behavior` de la entidad en el `ActorSystem` de cada nodo. Luego, el sistema de mensajería utiliza una referencia especial (`EntityRef`) para encontrar la ubicación de la entidad en el clúster.

```

1  import org.apache.pekko.cluster.sharding.typed.scaladsl.{ClusterSharding,
2  Entity, EntityTypeKey}
3
4  // Asumimos que 'system' es el ActorSystem con Cluster activado.
5  val system: ActorSystem[_] = ???
6
7  // 1. Definir la clave de la entidad
8  val TipoCuenta = EntityTypeKey[EntidadCuenta.Comando]("CuentaBancaria")
9
10 // 2. Inicializar el Cluster Sharding en el nodo.
11 ClusterSharding(system).init(
12   Entity(TipoCuenta) { entityContext =>
13     // entityContext.entityId es el ID lógico de la entidad (ej: "Cuenta-
14     // 123")
15     EntidadCuenta(entityContext.entityId)
16   }
17 )
18
19 // 3. Obtener una referencia de entidad para comunicación (EntityRef)
20 // Esto abstrae la ubicación real de la entidad en el clúster.
21 val cuentaRef = ClusterSharding(system).entityRefFor(TipoCuenta, "Cuenta-
22 ABC-123")
23
24 // Enviar un comando a la entidad, el sharding lo ruteará.
25 cuentaRef ! EntidadCuenta.Depositar(100.00, ...)

```

3. Ejemplos de Código: El Flujo de Event Sourcing

El ejemplo ilustra el flujo de comando a evento y la naturaleza de los *side effects* (efectos secundarios) en el `commandHandler`.

```
1  import org.apache.pekko.persistence.typed.scaladsl.Effect
2
3  // Componentes del Event Sourcing (predefinidos en la Sección 2A)
4
5  // --- EJEMPLO DE LÓGICA DE NEGOCIO ---
6
7  // 1. El actor recibe un comando (ej: Depositar)
8  // Llama al commandHandler: (Estado(500), Depositar(100, replyTo))
9
10 val commandHandlerEjemplo: (EntidadCuenta.Estado, EntidadCuenta.Comando)
    => Effect[EntidadCuenta.Evento, EntidadCuenta.Estado] =
11     (estado, comando) => comando match {
12       case EntidadCuenta.Depositar(monto, replyTo) =>
13         // Validación: (Si el monto es positivo, continúa)
14         if (monto > 0)
15           // Efecto 1: Persistir el evento (se envía al journal)
16           Effect.persist(EntidadCuenta.Depositado(monto))
17             .thenRun { nuevoEstado =>
18               // Side Effect (Ej: Logear o notificar)
19               println(s"Nuevo balance tras persistencia:
20                 ${nuevoEstado.balance}")
21             }
22           // Efecto 2: Responder al solicitante (solo después del
23           éxito de la persistencia)
24           .thenReply(replyTo)(_ => StatusReply.Success("OK"))
25       else
26         // Efecto 3: No persistir y solo responder con error (read-only
27         command)
28         Effect.reply(replyTo)(StatusReply.Error("Monto inválido"))
29     }
30
31 // 2. Si Effect.persist fue exitoso, se llama al eventHandler:
32 // (Estado(500), Depositado(100)) -> Nuevo Estado(600)
33
34 val eventHandlerEjemplo: (EntidadCuenta.Estado, EntidadCuenta.Evento) =>
    EntidadCuenta.Estado =
35     (estado, evento) => evento match {
36       case EntidadCuenta.Depositado(monto) =>
37         // Estado anterior: Estado(balance = 500)
38         // Nuevo estado: Estado(balance = 600)
39         estado.copy(balance = estado.balance + monto)
40     }
```

```

39 // 3. Recuperación: Si el actor se reinicia, el sistema aplica:
40 // emptyState (0) -> eventHandler(0, Depositado(100)) -> eventHandler(100,
    Depositado(50)) -> ...
41 // El estado se reconstruye secuencialmente.

```

4. Comparativa con Java y Python

La arquitectura de efectos puros y actores de Scala/Pekko ofrece una solución integrada para problemas de concurrencia y estado distribuido que en Java o Python requieren librerías de infraestructura externa y coordinación manual.

Aspecto	Java (Spring/JEE)	Python (Estructuras Dinámicas)	Scala (Pekko Actor Model)
Escalamiento de Estado	Alto acoplamiento con la infraestructura (Bases de datos, caches distribuidos). Requiere coordinación manual (ej. Redis, ZooKeeper).	N/A (Generalmente no se usa para sistemas distribuidos de estado).	Cluster Sharding: Distribución transparente y gestionada de entidades con estado, asegurando un escritor único (Single-Writer Principle).
Tolerancia a Fallos	Bloques <code>try-catch</code> y excepciones. La interrupción por excepción termina el hilo.	<code>try-except</code> .	Supervisión Jerárquica ("Let-It-Crash"): Los fallos se aíslan al actor y el supervisor decide reiniciar o reanudar.
Recuperación de Estado	Pérdida de estado en memoria en caso de reinicio de la aplicación o hilo.	Pérdida de estado en memoria.	Event Sourcing: El estado se reconstruye mediante la reproducción de eventos inmutables.
Resiliencia vs. Kubernetes	Kubernetes ofrece tolerancia a fallos a nivel de contenedor (coarse-grained). La pérdida de estado es un problema para aplicaciones <i>stateful</i> .	N/A	Sinergia: Pekko ofrece resiliencia a nivel de aplicación (fine-grained) (reinicio de actores/Fibers). Permite a los servicios <i>stateful</i> comportarse <i>como si fueran stateless</i> en Kubernetes.

5. Best Practices (*The Scala Way*)

1. **Modelado con Event Sourcing:** Para cualquier entidad cuyo estado deba sobrevivir a fallos o reinicios, utiliza `EventSourcedBehavior`. Este patrón es superior a la persistencia de estado duradero (Durable State) cuando se requiere trazabilidad y auditoría de la entidad.
2. **Pureza en `eventHandler`:** El `eventHandler` es la única fuente de verdad para el estado y **no debe contener efectos secundarios** (I/O, logs, llamadas a otros actores). La única responsabilidad del `eventHandler` es crear un **nuevo estado inmutable** a partir del estado antiguo y el evento.
3. **Encadenamiento de Efectos Secundarios:** Los efectos secundarios (como enviar una respuesta al remitente o logear una acción) deben encadenarse **después** de la persistencia del evento exitosa, utilizando combinadores como `Effect.persist(...).thenRun(...)` o `Effect.thenReply(...)`.
4. **Cluster Sharding para el Escalado:** Si la aplicación se ejecuta en un clúster, el uso de `ClusterSharding` es idiomático para distribuir la carga y garantizar el **Single-Writer Principle** necesario para la integridad de `EventSourcedBehavior`.

La Persistencia Reactiva y los Sistemas Distribuidos de Pekko son una manifestación del principio de **inmutabilidad aplicada a la infraestructura**. Al tratar los cambios de estado como un registro inmutable (Event Sourcing) y al distribuirlos bajo la estricta vigilancia del Cluster Sharding, se construye un sistema que no teme a los fallos, sino que los utiliza como una oportunidad para demostrar su capacidad de **auto-recuperación y escalabilidad**.